



طراحی و پیاده سازی واحد حساب و منطق 32 بیتی معماری کامپیوتر - گزارش کار تمرین عملی اول

استاد
دکتر سربازی آزاد

دانشگاه صنعتی شریف
بهار 1405

محمد مهدی مرادی - 403106681

فهرست

3.....	مقدمه
3.....	مشخصات ورودی ها و خروجی ها
4.....	نحوه طراحی
4.....	1-ورودی ها
5.....	2-جمع کننده/تفریق کننده 32بیتی
6.....	3-پرچم ها
7.....	4-ضرب کننده 32 بیتی بدون علامت
10.....	5-خروجی ها

مقدمه:

در این تمرین قصد داریم که یک واحد حساب و منطق ترکیبی 32 بیتی را درون نرم افزار لاجیسیم طراحی و پیاده سازی کنیم. این واحد قادر به انجام جمع و تفریق بدون علامت و باعلامت است و همچنین میتواند اعداد بدون علامت را نیز ضرب کند و در 64 بیت نشان دهد.

مشخصات ورودی ها و خروجی ها:

ورودی ها:

- A: عملوند اول (32 بیت)
- B: عملوند دوم (32 بیت)
- ALU_Ctrl: سیگنال کنترلی (3 بیت)

خروجی ها:

- Result_Lo: 32 بیت پایین حاصل عملیات
- Result_Hi : 32 بیت بالا حاصل عملیات که فقط هم در ضرب کاربرد دارد و در بقیه حالات صفر است.
- C(carry): پرچم نقلی
- V(overflow): پرچم سرریز
- Z(zero): پرچم صفر بودن حاصل عملیات
- N(negative): پرچم منفی بودن حاصل عملیات

در جدول 1 هر 8 حالت سیگنال کنترلی مدار (ALU_Ctrl) را میبینیم.

جدول ۱: کدهای کنترلی واحد حساب و منطق

کد کنترلی ^۱	نام دستور	شرح عملیات	پرچم های فعال
000	ADD_U	جمع دو عدد بدون علامت ($A + B$)	C, Z, V
001	ADD_S	جمع دو عدد علامت دار ($A + B$)	V, N, Z
010	SUB_U	تفریق دو عدد بدون علامت ($A - B$)	C, Z, V
011	SUB_S	تفریق دو عدد علامت دار ($A - B$)	V, N, Z
100	MUL_U	ضرب ترکیبی دو عدد بدون علامت ($A \times B$)	Z
سایر حالت ها	NOP	خروجی ها صفر باشند.	هیچ کدام

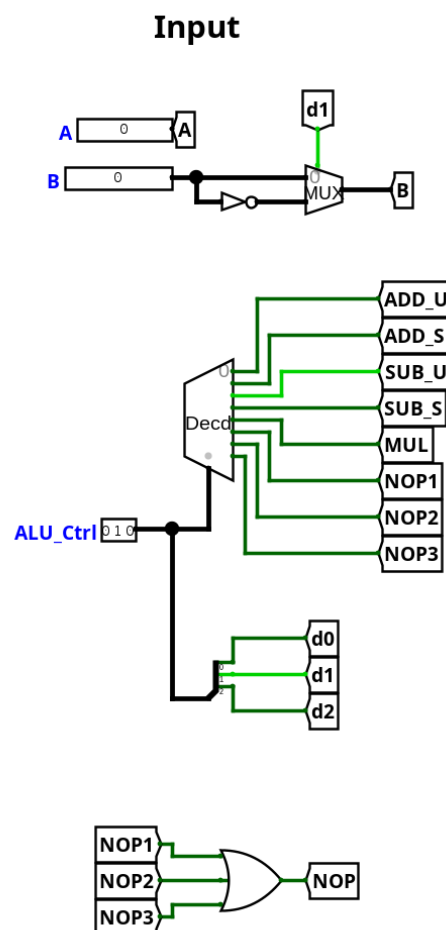
نحوه طراحی:

1- ورودی ها:

برای هر 3 ورودی مدار با استفاده از پین های داخل لاجیسیم میاییم و ورودی هارا به این پین ها متصل میکنیم. توجه کنید که پین متصل به عملوند ها 32 بیتی هستند ولی برای سهولت انجام تست ها نمایش دهندهی دارند و پین متصل به سیگنال کنترلی همان نمایش باینری را دارد. در این تمرین استفاده زیادی از تونل ها شده تا حداقل سیم کشی را در مدار خود داشته باشیم و مدار تمیز و قابل فهم تر باشد.

عملوند دوم (B) را باید به یک مولتی پلکسر متصل کنیم که خط انتخاب آن باید از بیت دوم سیگنال کنترلی بیاید، چون که در حالت هایی که بیت دوم مدار ما عملیات تفریق را انجام میدهد؛ پس ورودی صفر را باید همان ورودی B و ورودی یک را نات B بگذاریم.

سیگنال کنترلی را با استفاده از یک رمزگشا 3 در 8 میاییم و هر 8 حالت مدار خود را به تونل های مربوطه متصل میکنیم. همچنین برای حالت 3، 101، 110، 111 که مدار با هیچ عملیاتی را نباید انجام دهد، میاییم و 3 خروجی پایین رمزگشا را به یک گیت or میدهم تا هروقت این سیگنال یک شد یعنی مدار نباید هیچ کاری بکند و همه ی خروجی ها باید صفر باشد. پیاده سازی تمامی ورودی هارا در شکل 1 مشاهده میکنید.



شکل 1

2- جمع کننده/تفریق کننده 32 بیتی:

برای طراحی این قطعه میایم مشابه با طراحی یک full adder عمل میکنیم و به این صورت که:

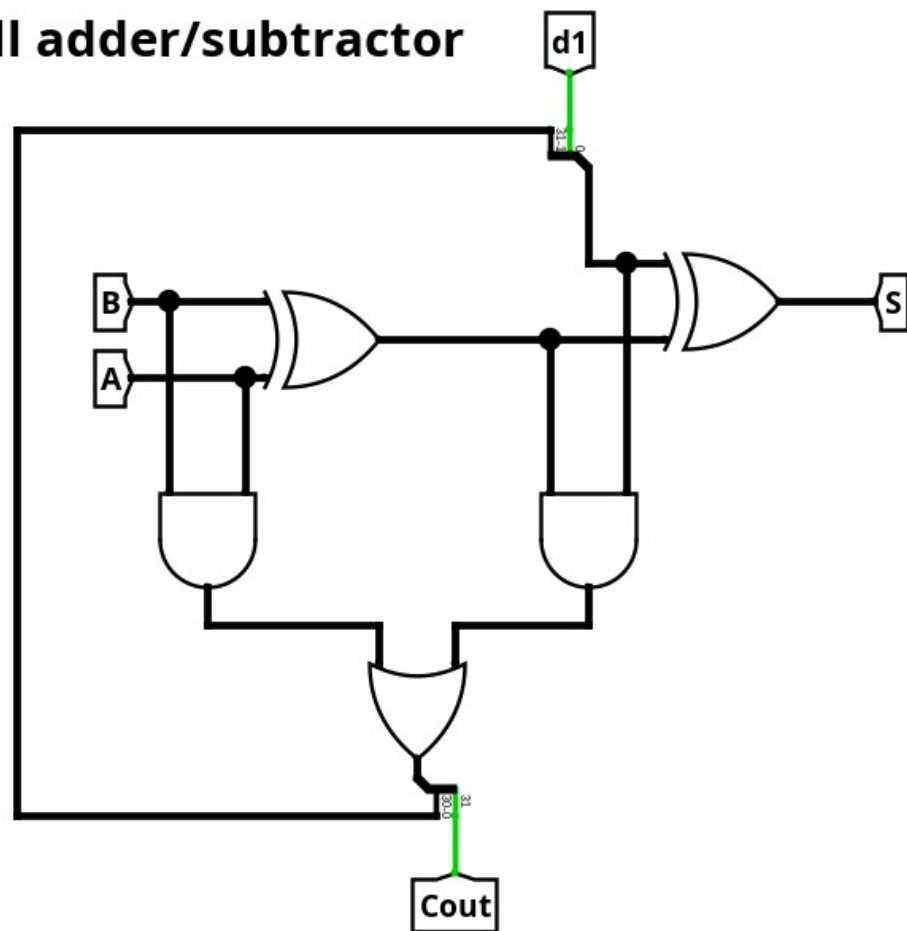
$$S = A \text{ xor } B \text{ xor } \text{Cin}$$

$$\text{Cout} = AB + AC_{\text{in}} + BC_{\text{in}}$$

تنها فرق این قطعه این است که باید بیت های 30-0 Cout را به بیت های 1-31 Cin وصل کنیم و بیت 0 کری ورودی هم باید از بیت دوم سیگنال کنترلی بیاید که در حالت تفریق عملوند دوم معادل مکمل دو بشود.

مدار نهایی را در شکل 2 مشاهده میکنید.

32-bit full adder/subtractor



شکل 2

3-پرچم ها:

در تمامی پرچم ها قبل از دادن خروجی نهایی سیگنالی که میگوید مدار در حالت NOP(no operation) است را با خروجی نهایی and میکنیم تا اگر قرار بود مدار کاری نکند خروجی ها همه صفر شود.

3-1-پرچم Z:

برای طراحی این پرچم باید با استفاده از یک اسپلیتر هر 32 بیت خروجی را جدا کنیم و به یک گیت 32 or ورودی بدهیم. باید نات پرچم C نیز and کنیم به این دلیل که اگر بیت 33 یک شد و تمامی 32 بیت دیگر صفر شد در اصل این نباید پرچم صفر را فعال کند پس این را هم and میکنیم تا از این اتفاق جلوگیری شود.

3-2-پرچم C:

برای طراحی این قسمت باید ابتدا توجه داشته باشیم که ما باید رد حالت های جمع یا تفریق بدون علامت باشیم پس در ابتدا نات بیت های اول و سوم سیگنال کنترلی را با هم and میکنیم. سپس باید چک کنیم که آیا در حالت تفریق هستیم یا نه. اگر باشیم باید وقتی cout صفر میشود این پرچم فعال شود و اگر در حالت جمع باشیم وقتی یک میشود باید این پرچم فعال شود. برای این موضوع میاییم و بیت دوم سیگنال کنترلی را که نشان دهنده جمع یا تفریق است را با cout ایکسور میکنیم. در نهایت همه را با هم and میکنیم و به تونل پرچم C متصل میکنیم.

3-3-پرچم V:

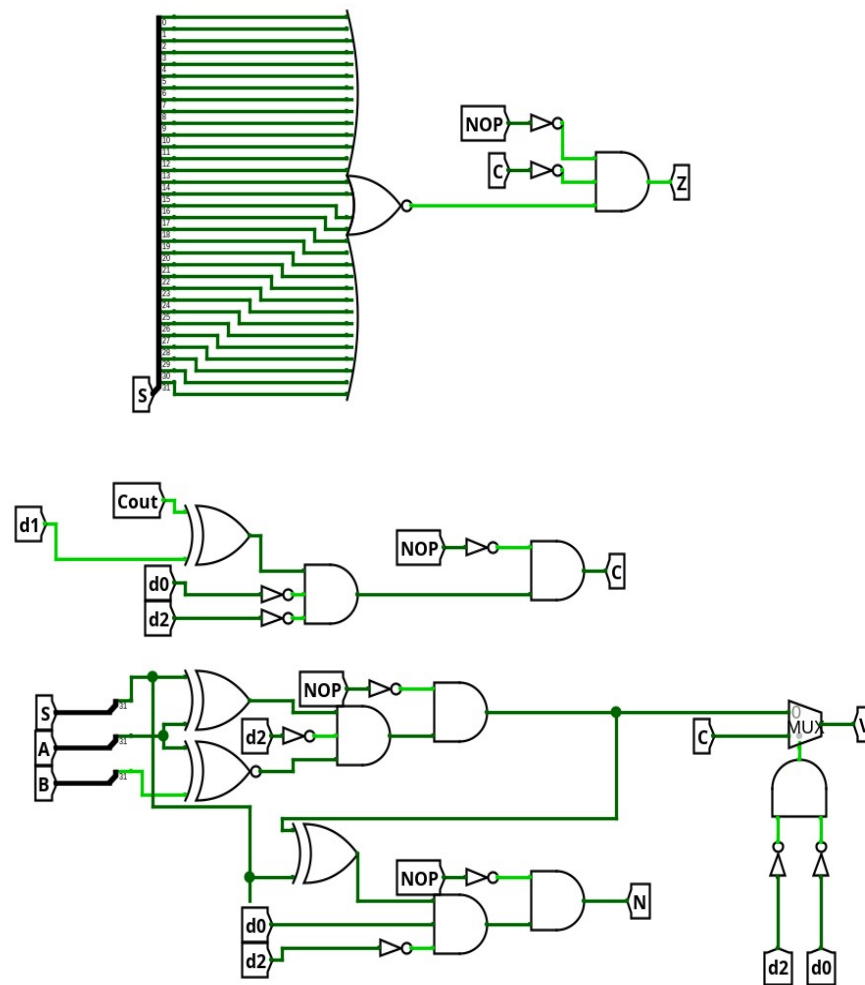
برای پرچم اورفلو باید این را بدانیم که اورفلو وقتی رخ میدهد که دو عدد هم علامت را جمع یا تفریق کنیم و علامت حاصل متفاوت شود. بدین منظور باید یکسان بودن علامت دو عملوند را چک کنیم و متفاوت بودن آن را با علامت خروجی نیز چک بکنیم. برای اینکار میاییم بیت آخر دو عملوند و خروجی را با یک اسپلیتر جدا میکنیم، دو بیت آخر عملوند ها را با یکدیگر xnor و یکی از بیت های آخر عملوند ها را با بیت آخر خروجی xor میکنیم. در آخر هم باید به این نکته توجه کنیم که اگر در حالت جمع یا تفریق بدون علامت باشیم، پرچم C و پرچم V مقدار یکسانی با هم خواهند داشت پس باید با یک مولتی پلکسر این را هم پیاده بکنیم.

3-4-پرچم N:

برای این پرچم یک بودن بیت آخر جواب کافی نیست، بلکه باید توجه کنیم که اگر بیت آخر صفر بود و پرچم کری یک بود باز هم عدد منفی است و پرچم N باید فعال شود، برای انجام اینکار باید بیت آخر جواب و پرچم کری را با هم xor بکنیم.

تمامی طراحی ها را در شکل 3 مشاهده میکنید.

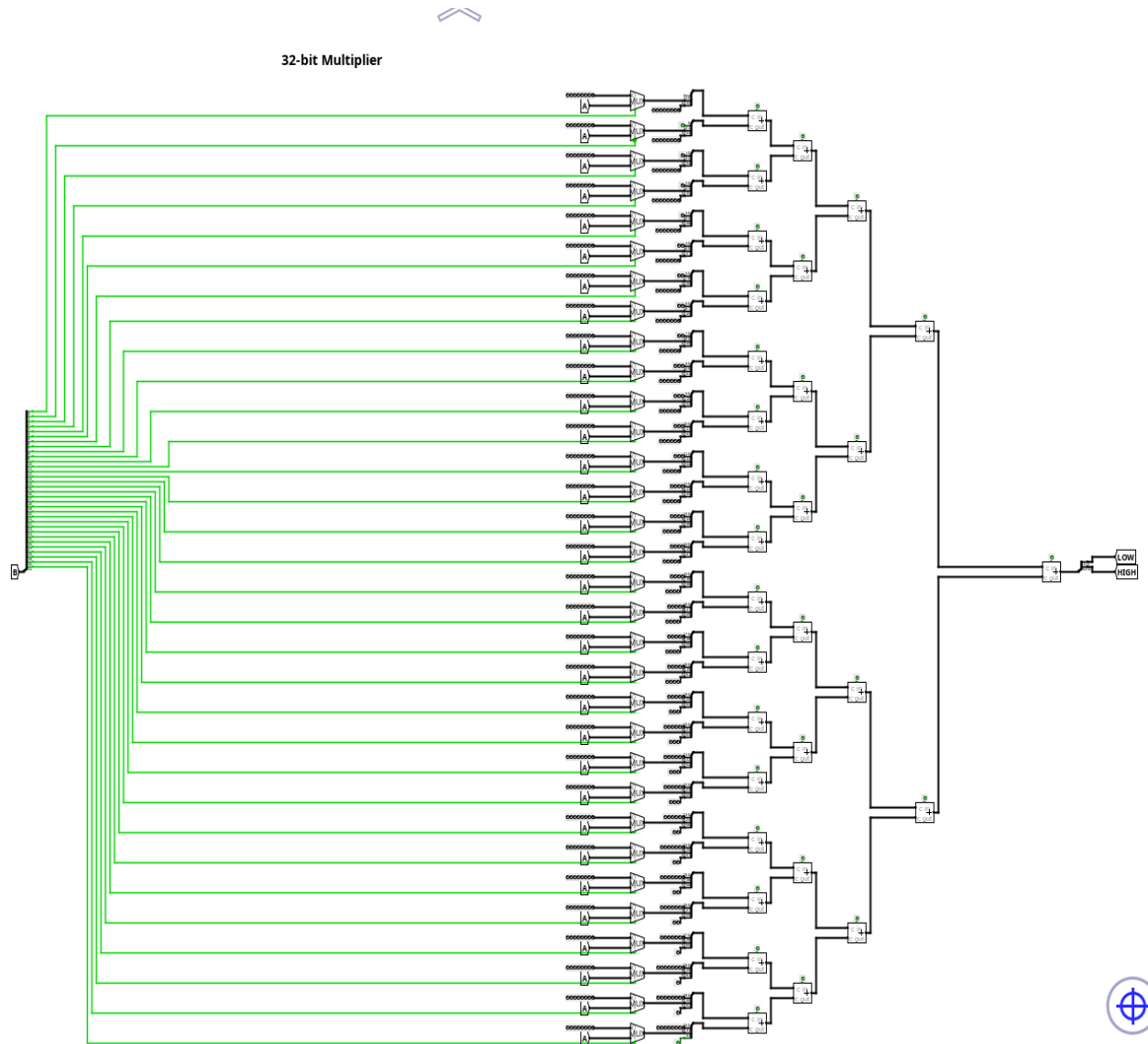
Flags



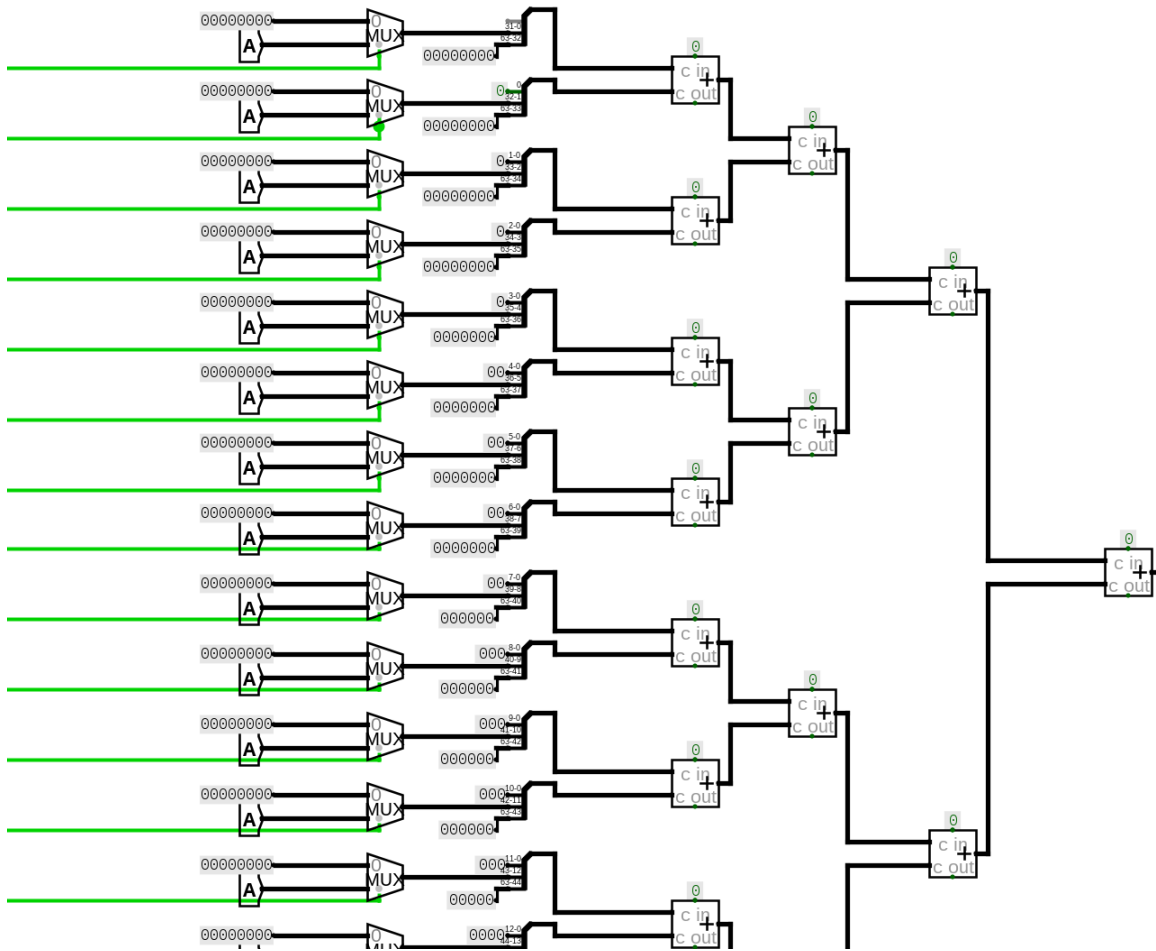
شکل 3

4- ضرب کننده 32 بیتی بدون علامت:

برای طراحی این ضرب کننده مانند کاری که روی کاغذ انجام می‌دهیم عمل می‌کنیم. اینجوری که تمامی بیت های عملوند دوم را با اسپلیتر جدا می‌کنیم، هر کدام از 32 بیت را به خط انتخاب یک مولتی پلکسر 2 در 1 در 32 می‌دهیم، ورودی صفر مولتی پلکسر را به یک ثابت صفر می‌دهیم و ورودی یک را به عملوند اول می‌دهیم. با این کار داریم می‌گوییم که اگر بیت i ام B یک بود عدد A و اگر صفر بود عدد 32 بیتی صفر را انتخاب می‌کنیم. در مرحله بعد باید این 32 عدد 32 بیتی را در ابتدا به 64 بیت گسترش دهیم (چون ضرب دو عدد 32 بیتی حداکثر در 64 بیت جا می‌شود) و سپس به اندازه جایگاه مکانی بیت i ام، این عدد را i تا شیفت به چپ بدهیم (این عمل را روی کاغذ نیز انجام می‌دهیم). برای پیاده کردن این تیکه باید با استفاده از یک اسپلیتر که برعکس گذاشته شده، بیت های 0 تا $i-1$ را به یک ثابت صفر بدهیم (همان عمل شیفت به چپ) و بیت های i تا $i+31$ را به همان خروجی گرفته شده از مولتی پلکسر بدهیم و بیت های $i+32$ تا 63 را باز هم به یک ثابت صفر بدهیم (همان عمل گسترش به 64 بیت). در آخر ما 32 عدد 64 بیتی داریم که به صورت آبشاری با استفاده از 31 عدد جمع کننده 64 بیتی خروجی نهایی را تولید می‌کنیم. در آخر هم با استفاده از یک اسپلیتر خروجی را به دو 32 بیت تقسیم می‌کنیم (برای $Result_Hi$ و $Result_Lo$). طراحی این قطعه را در شکل های 4 و 5 مشاهده می‌کنید.



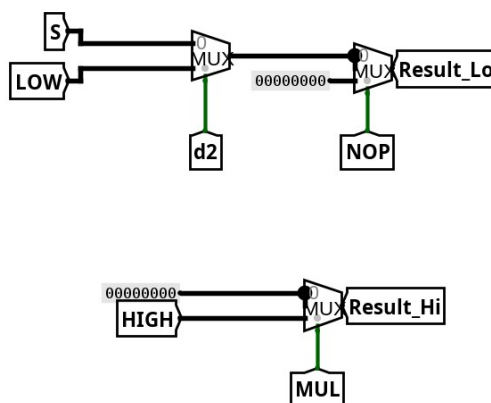
شکل 4



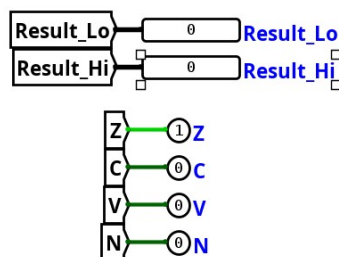
شکل 5

5-خروجی ها:

در این قسمت فقط دو نکته وجود دارد. 1- باید با استفاده از یک مولتی پلکسر 2 در 1 در 32 و با خط انتخابی که به بیت سوم سیگنال کنترلی وصل شده، بیاییم و مشخص کنیم که اگر این بیت یک بود حاصل 32 بیت پایین ضرب کننده را به خروجی بدهیم و اگر صفر بود یعنی در حالت ضرب نیستیم و خروجی جمع کننده/تفریق کننده را باید به خروجی بدهیم. همانند بخش پرچم ها، باید حواسمان به این باشد که اگر در حالت NOP بودیم باید خروجی صفر شود که این را هم با استفاده از یک مولتی پلکسر 2 در 1 در 32 پیاده میکنیم. 2- برای 32 بیت بالا تنها در زمانی که در حالت ضرب بودیم باید این خروجی از 32 بیت بالای حاصل ضرب کننده بیاید در غیر این صورت باید صفر باشد، برای این کار نیز یک مولتی پلکسر 2 در 1 در 32 میگذاریم. در آخر برای نمایش دادن هر 6 خروجی، از پین هایی که حالت خروجی دارند استفاده میکنیم. (در ابتدای گزارش اشاره شد که با تونل ها زیاد کار شده است و در آخر هم برای خروجی دادن این تونل ها را به پین های مربوطه متصل میکنیم) در شکل 6 نحوه طراحی این خروجی ها را میبینید.



Output



شکل 6

در شکل 7 نیز خروجی تست مدار که توسط tb.v انجام شده است را مشاهده میکنید.

```
mahdi@mahdi-ASUS-Zenbook-14:~/Documents/CA/HWS/Practicals/ca4042-judge$ ./synth_valid.sh ../HW1-403106681.circ ../HW1/tb.v
logisim workspace : /home/mahdi/logisim_evolution_workspace
inpins : ['ALU_Ctrl', 'A', 'B']
outpins : ['Result_Lo', 'Result_Hi', 'Z', 'C', 'V', 'N']
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Building netlist for sheet "main"
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Circuit "main" has 66 nets and 112 buses.
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Circuit "main" passed DRC check.
[main] INFO com.cburch.logisim.fpga.gui.Reporter - The Board ALCHITRY_AU_IO has:
[main] INFO com.cburch.logisim.fpga.gui.Reporter - 6 Button(s)
[main] INFO com.cburch.logisim.fpga.gui.Reporter - 3 DIPSwitch(s)
[main] INFO com.cburch.logisim.fpga.gui.Reporter - 32 Led(s)
[main] INFO com.cburch.logisim.fpga.gui.Reporter - 1 SevenSegmentScanning(s)
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating HDL file: /home/mahdi/logisim_evolution_workspace/HW1-403106681.circ.tmp/main/verilog/gates/AND_GATE_BUS.v
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating HDL file: /home/mahdi/logisim_evolution_workspace/HW1-403106681.circ.tmp/main/verilog/gates/XOR_GATE_ONEHOT.v
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating HDL file: /home/mahdi/logisim_evolution_workspace/HW1-403106681.circ.tmp/main/verilog/gates/XNOR_GATE_ONEHOT.v
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating HDL file: /home/mahdi/logisim_evolution_workspace/HW1-403106681.circ.tmp/main/verilog/gates/XOR_GATE_BUS_ONEHOT.v
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating HDL file: /home/mahdi/logisim_evolution_workspace/HW1-403106681.circ.tmp/main/verilog/gates/OR_GATE_BUS.v
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating HDL file: /home/mahdi/logisim_evolution_workspace/HW1-403106681.circ.tmp/main/verilog/gates/AND_GATE_3_INPUTS.v
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating HDL file: /home/mahdi/logisim_evolution_workspace/HW1-403106681.circ.tmp/main/verilog/gates/NOR_GATE_32_INPUTS.v
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating HDL file: /home/mahdi/logisim_evolution_workspace/HW1-403106681.circ.tmp/main/verilog/gates/AND_GATE.v
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating HDL file: /home/mahdi/logisim_evolution_workspace/HW1-403106681.circ.tmp/main/verilog/gates/OR_GATE_3_INPUTS.v
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating HDL file: /home/mahdi/logisim_evolution_workspace/HW1-403106681.circ.tmp/main/verilog/plexers/Multiplexer_bus_2.v
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating HDL file: /home/mahdi/logisim_evolution_workspace/HW1-403106681.circ.tmp/main/verilog/plexers/Multiplexer_2.v
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating HDL file: /home/mahdi/logisim_evolution_workspace/HW1-403106681.circ.tmp/main/verilog/plexers/Decoder_8.v
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating HDL file: /home/mahdi/logisim_evolution_workspace/HW1-403106681.circ.tmp/main/verilog/arith/Adder.v
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating HDL file: /home/mahdi/logisim_evolution_workspace/HW1-403106681.circ.tmp/main/verilog/circuit/main.v
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating HDL file: /home/mahdi/logisim_evolution_workspace/HW1-403106681.circ.tmp/main/verilog/topLevel/logisimTopLevelShe
ll.v
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating script file: /home/mahdi/logisim_evolution_workspace/HW1-403106681.circ.tmp/main/scripts/vivadoCreateProject.tcl
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating script file: /home/mahdi/logisim_evolution_workspace/HW1-403106681.circ.tmp/main/scripts/vivadoGenerateBitStream.
tcl
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating script file: /home/mahdi/logisim_evolution_workspace/HW1-403106681.circ.tmp/main/scripts/vivadoLoadBitStream.tcl

../HW1/tb.v /home/mahdi/logisim_evolution_workspace/HW1-403106681.circ.tmp
../HW1/tb.v:34: Warning: Calling system function $random() as a task.
../HW1/tb.v:34: The functions return value will be ignored.
ADD_U
SUB_U
ADD_S
SUB_S
MUL_U
ACCEPTED
50 / 50
```

شکل 7